

Logic in AI : Reduction to Satisfiability Problems - Part - 6

Introduction

Up until this point, our focus in logic has been on inference: given a set of formulas, can we logically prove a particular statement? This involved strategies for deducing new truths from existing ones. However, the landscape of logical problems extends beyond mere proof. This lecture introduces a pivotal shift towards "model finding," a distinct yet equally crucial area in artificial intelligence. Model finding seeks to discover an assignment of truth values to variables that makes a given logical formula true. This is fundamentally about finding a "satisfying assignment" or a "model" for a formula, rather than proving its validity. This seemingly simple task underpins the "satisfiability problem," a canonical NP-complete problem with profound implications across computer science and AI.

The satisfiability problem, often abbreviated as SAT, is not just a theoretical construct; it is a powerful tool for solving a vast array of real-world computational challenges. Its importance stems from the fact that many complex problems can be "reduced" to SAT, meaning they can be reformulated as a SAT problem such that a solution to the SAT problem directly provides a solution to the original problem. The remarkable efficiency of modern SAT solvers has transformed them into indispensable instruments for tasks ranging from circuit verification and optimization to planning, scheduling, and even diagnosing malfunctioning systems. Understanding how to translate diverse problems into the language of SAT is a critical skill for anyone working in AI and computational problem-solving.

Core Concepts

Term	Definition	Significance
Model Finding	The process of identifying an assignment of truth values to variables that makes a given logical formula true.	Shifts focus from proving statements to finding concrete solutions or configurations that satisfy conditions.
Satisfiability Problem (SAT)	Given a logical formula, determine if there exists at least one assignment of truth	A foundational problem in computer science, it is the canonical NP-

Term	Definition	Significance
	values to its variables that makes the formula true.	complete problem, central to many computational tasks.
NP-Complete	A class of decision problems that are the "hardest" problems in NP. If a polynomial-time algorithm exists for any NP-complete problem, then all problems in NP can be solved in polynomial time.	Highlights the computational complexity of SAT and its role as a benchmark for problem-solving efficiency.
Non-deterministic Polynomial Time (NP)	Refers to problems whose solutions can be verified in polynomial time by a deterministic Turing machine, or computed in polynomial time by a non-deterministic Turing machine.	Crucial for understanding the theoretical limits and practical approaches to solving complex problems.
Conjunctive Normal Form (CNF)	A standardized way to represent logical formulas as a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals (variables or their negations).	The standard input format for most modern SAT solvers, simplifying the problem structure for efficient processing.
Circuit Equivalence	Determining if two different logical circuits compute the exact same function for all possible inputs.	A vital application of SAT in hardware design and verification, ensuring correctness after optimization.
N-Queens Problem	Placing N non-attacking queens on an N×N chessboard.	A classic combinatorial problem used to illustrate how constraints can be encoded into SAT formulas.

Term	Definition	Significance
Graph Coloring Problem	Assigning colors to vertices of a graph such that no two adjacent vertices share the same color, using a minimum number of colors.	Another classic problem demonstrating the reduction of Constraint Satisfaction Problems (CSPs) to SAT.
Model-Based Diagnosis	Diagnosing faults in a system by comparing observed behavior with a logical model of how the system is supposed to work.	A powerful application of SAT in fault detection and repair for complex systems, like spacecraft.

Detailed Analysis

1. Model Finding and NP-Completeness

The core distinction between inference and model finding lies in their objectives. Inference aims to prove a statement's truth, while model finding seeks a specific assignment of variables that makes a formula true. This latter task is precisely what the Satisfiability Problem (SAT) addresses. SAT is renowned as a canonical NP-complete problem. Understanding NP-completeness is crucial:

* **NP (Non-deterministic Polynomial time)**: Problems in NP are those for which a given solution can be verified in polynomial time. It does not mean "not polynomial time." If a problem is in P (polynomial time), it is also in NP. The P vs. NP question, a billion-dollar problem in computer science, asks whether every problem whose solution can be quickly verified can also be quickly solved.

* **NP-Complete**: These are the hardest problems in NP. If you can find a polynomial-time solution for any NP-complete problem, you can find polynomial-time solutions for all problems in NP. SAT is one such problem, making it a central focus for algorithmic research. While finding a solution for NP-complete problems is generally exponential, verifying a proposed solution is polynomial. Modern SAT solvers, despite the theoretical hardness, are "blazingly fast" in practice for many instances.

2. Circuit Equivalence Verification

A practical application of SAT is verifying if two complex circuits compute the same function. This is critical in circuit optimization, where a designer might simplify a circuit and needs to ensure the optimized version is functionally identical to the original.

- * **Problem Formulation:** Given two circuits, say one producing output C and another C' , we want to know if C is always equivalent to C' .
- * **SAT Reduction Strategy:** Instead of directly asking if $C = C'$, we ask: "Is there any input assignment for which C is not equal to C' ?" If such an assignment exists, the circuits are not equivalent. If no such assignment can be found (i.e., the SAT problem is unsatisfiable), then the circuits are equivalent.
- * **Logical Encoding:**
 - * Define the logic of each circuit using propositional variables. For example, if C is $A \text{ AND } B$, and C' is $\text{NOT } D \text{ OR } E$ where D is $\text{NOT } A$ and E is $\text{NOT } B$:
 - * $C \iff (A \text{ AND } B)$
 - * $C' \iff (\text{NOT } D \text{ OR } E)$
 - * $D \iff (\text{NOT } A)$
 - * $E \iff (\text{NOT } B)$
 - * Add a clause representing the non-equivalence: $\text{NOT } (C \iff C')$. This can be expanded into $(C \text{ AND } \text{NOT } C') \text{ OR } (\text{NOT } C \text{ AND } C')$.
 - * The SAT solver then attempts to find an assignment for A, B, C, C', D, E that satisfies all these clauses. If it finds one, the circuits are not equivalent. If it finds no such assignment, they are equivalent.

3. The N-Queens Problem

The N-Queens problem (e.g., 8 queens on an 8x8 board) is a classic example of encoding combinatorial constraints into SAT.

- * **Variables:** For an $N \times N$ board, define $N \times N$ propositional variables Q_{ij} , where Q_{ij} is true if a queen is placed in row i and column j , and false otherwise. For an 8x8 board, this means 64 variables.
- * **Constraints Encoding:**
 - * **No two queens attack each other:** This is the primary challenge. For any two positions (i_1, j_1) and (i_2, j_2) that attack each other (same row, column, or diagonal), we must ensure that $Q_{i_1 j_1}$ and $Q_{i_2 j_2}$ are not both true. This is expressed as $\text{NOT } (Q_{i_1 j_1} \text{ AND } Q_{i_2 j_2})$, which is equivalent to $(\text{NOT } Q_{i_1 j_1} \text{ OR } \text{NOT } Q_{i_2 j_2})$. This clause must be generated for all attacking pairs.
 - * **Every row must have exactly one queen:**
 - * At least one queen per row: For each row i , $(Q_{i1} \text{ OR } Q_{i2} \text{ OR } \dots \text{ OR } Q_{iN})$.
 - * At most one queen per row: For each row i and any two distinct columns j_1 and j_2 , $(\text{NOT } Q_{i j_1} \text{ OR } \text{NOT } Q_{i j_2})$.
 - * **Every column must have exactly one queen:** Similar to rows.
 - * At least one queen per column: For each column j , $(Q_{1j} \text{ OR } Q_{2j} \text{ OR } \dots \text{ OR } Q_{Nj})$.
 - * At most one queen per column: For each column j and any two distinct rows i_1 and i_2 , $(\text{NOT } Q_{i_1 j} \text{ OR } \text{NOT } Q_{i_2 j})$.

* A satisfying assignment to these Q_{ij} variables provides a valid solution to the N-Queens problem.

4. Map Coloring Problem (CSP to SAT)

The map coloring problem, a type of Constraint Satisfaction Problem (CSP), can also be reduced to SAT. This demonstrates that any discrete, finite CSP can be converted into a SAT formula.

* **Variables:** For each state S and each possible color C , define a propositional variable X_{S_C} . For example, X_{WA_red} is true if Western Australia is colored red. If there are 7 states and 3 colors, this results in 21 variables.

* **Constraints Encoding:**

* **Every state must have at least one color:** For each state S , $(X_{S_color1} \text{ OR } X_{S_color2} \text{ OR } \dots \text{ OR } X_{S_colorK})$. E.g., $(X_{WA_red} \text{ OR } X_{WA_green} \text{ OR } X_{WA_blue})$.

* **Every state must have at most one color:** For each state S and any two distinct colors $C1$ and $C2$, $(\text{NOT } X_{S_C1} \text{ OR } \text{NOT } X_{S_C2})$. This prevents a state from being assigned multiple colors simultaneously. E.g., $(\text{NOT } X_{WA_red} \text{ OR } \text{NOT } X_{WA_green})$.

* **Neighboring states must have different colors:** For any two adjacent states $S1$ and $S2$, and for each color C , $(\text{NOT } X_{S1_C} \text{ OR } \text{NOT } X_{S2_C})$. E.g., if Western Australia (WA) and Northern Territory (NT) are neighbors, then $(\text{NOT } X_{WA_red} \text{ OR } \text{NOT } X_{NT_red})$.

* A satisfying assignment to these X_{S_C} variables provides a valid coloring for the map.

5. Model-Based Diagnosis

SAT solvers are used in diagnosing malfunctioning devices, a field known as model-based diagnosis.

* **System Model:** The device's design and functionality are expressed as a logical formula F . This formula describes how components interact and what constitutes normal operation.

* **Observations:** We have a set of observed states O (e.g., "engine is not running," "gas in tank," "clock is running").

* **Assumptions:** We assume all components are working correctly, represented by a set of propositions A (e.g., "fuel line OK," "battery OK," "cables OK").

* **Diagnosis Process:**

1. Initially, check if $F \text{ AND } O \text{ AND } A$ is consistent (satisfiable). If the engine is not running but all components are assumed OK, this formula will likely be unsatisfiable.
2. If inconsistent, it means at least one assumption in A is false. A diagnosis is a minimal set of assumptions that, when negated and added to $F \text{ AND } O$, makes the overall formula consistent.

3. To find a diagnosis, propositions are systematically removed (or negated) from A one at a time, and consistency is re-checked. For example, if $F \text{ AND } 0 \text{ AND } (A - \{fuel \ line \ OK\}) \text{ AND } \{NOT \ fuel \ line \ OK\}$ is satisfiable, then "fuel line not OK" is a possible diagnosis.

* **Deep Space One Example:** NASA successfully used a SAT solver for diagnosing faults in the Deep Space One spacecraft. The entire system schematic was compiled into a 7000-variable SAT problem. The system, called Remote Agent, was able to diagnose and compensate for simulated faults, such as a failed electronics unit, a faulty sensor, or a stuck thruster. This highlights the mission-critical applications of SAT solvers.

Key Takeaways

- [] Model finding aims to find variable assignments that make a formula true, distinct from inference which proves statements.
- [] The Satisfiability Problem (SAT) is the canonical NP-complete problem, meaning many other complex problems can be reduced to it.
- [] NP stands for Non-deterministic Polynomial time, indicating that solutions can be verified in polynomial time, not that they are "not polynomial."
- [] Complex problems like circuit equivalence, N-Queens, and map coloring can be systematically converted into SAT formulas.
- [] For circuit equivalence, the problem is reframed as finding an input where the circuits are not equivalent.
- [] Encoding N-Queens requires variables for queen placement and clauses for non-attacking conditions and ensuring queens in every row/column.
- [] Converting CSPs to SAT involves defining variables for each (variable, value) pair and encoding domain, uniqueness, and constraint clauses.
- [] Model-based diagnosis uses SAT to find minimal sets of faulty components consistent with observed system behavior.
- [] Modern SAT solvers are remarkably efficient in practice, enabling their use in critical applications like spacecraft diagnosis.

Conclusion

The journey from logical inference to model finding, culminating in the satisfiability problem, reveals a powerful paradigm shift in computational problem-solving. SAT is not merely a theoretical curiosity but a practical workhorse, capable of tackling problems that span diverse domains. Its status as an NP-complete problem underscores its inherent complexity, yet the continuous advancements in SAT solver

technology have made it an indispensable tool for engineers and AI researchers alike.

The ability to reduce complex real-world challenges—from verifying the correctness of microchips to diagnosing faults in autonomous spacecraft—into a SAT problem demonstrates the profound utility of this logical framework. As systems become more intricate and the demand for robust, verifiable solutions grows, the skill of modeling problems for SAT solvers will only become more critical. This lecture has laid the groundwork for understanding not just what SAT is, but how to leverage its power by translating diverse problem constraints into its universal language, paving the way for efficient and automated solutions in the age of AI.